Article title: GraphRAG and Knowledge Grounding for Agentic Systems in SAP Environments
Authors: Morris Campbell[1], Harry Michaels[1]
Affiliations: university of illinois at springfield[1]
Orcid ids: 0009-0005-7758-5734[1], 0009-0000-7565-4541[1]
Contact e-mail: kenzieharryizevbekhai@gmail.com
License information: This work has been published open access under Creative Commons Attribution License http://creativecommons.org/licenses/by/4.0/, which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited. Conditions, terms of use and publishing policy can be found at https://www.scienceopen.com/.
Preprint statement: This article is a preprint and has not been peer-reviewed, under consideration and submitted to ScienceOpen Preprints for open peer review.
DOI: 10.14293/PR2199.004145.v1
Preprint first posted online: 22 July 2026
Keywords: GraphRAG, Knowledge Grounding, Agentic Systems, Enterprise AI, SAP Environments, Large Language Models, Retrieval-Augmented Generation, Hallucination Mitigation

GraphRAG and Knowledge Grounding for Agentic Systems in SAP Environments

Morris Campbell[1], Harry Michaels[2]

[1]University of Illinois at Springfield

[2]University of Illinois at Springfield

July 2026

Abstract

The integration of Large Language Models (LLMs) into enterprise systems promises transformative automation but is hindered by inherent challenges such as hallucination, lack of domain-specific context, and opaque reasoning processes. In complex, regulated environments like SAP landscapes—characterized by intricate relational data models, proprietary business logic, and dynamic transactional workflows—these limitations pose significant operational risks. This research study investigates the synergistic application of GraphRAG (Graph-based Retrieval-Augmented Generation) and advanced knowledge grounding techniques to enhance the reliability, transparency, and adaptability of agentic AI systems within SAP environments. We propose a novel architectural framework that leverages SAP's semantic metadata, process hierarchies, and master data relationships to construct a dynamic enterprise knowledge graph. This graph serves as the foundational retrieval corpus, enabling a multi-agent system to ground its reasoning and action generation in verifiable, context-aware factual data. Through a series of controlled experiments simulating common SAP use cases—including intelligent procurement, automated order-to-cash processing, and dynamic master data governance—we evaluate the system's performance against baseline RAG models and fine-tuned LLMs. Preliminary findings indicate that our GraphRAG approach significantly reduces factual errors, improves traceability of AI decisions to source data, and enhances the agents' ability to handle multi-step, cross-module queries. Furthermore, we introduce a feedback loop for continuous knowledge updating, ensuring alignment with real-time system states. This study contributes a validated reference architecture and empirical evidence for deploying trustworthy, autonomous agents in enterprise resource planning systems, bridging the gap between generative AI capabilities and the rigorous demands of operational SAP landscapes.

Keywords: GraphRAG, Knowledge Grounding, Agentic Systems, Enterprise AI, SAP Environments, Large Language Models, Retrieval-Augmented Generation, Enterprise Knowledge Graphs, Process Automation, Hallucination Mitigation, Semantic Reasoning, Multi-Agent Systems

1. Introduction

The enterprise software landscape is undergoing a profound transformation driven by the rapid advancement of generative artificial intelligence, particularly large language models. These models have demonstrated remarkable capabilities in natural language understanding, reasoning, and content generation, opening new possibilities for automating complex business processes. However, the deployment of LLMs in enterprise environments presents significant challenges that cannot be addressed through model capabilities alone (Gao et al., 2023). The limitations of these models—including their tendency to hallucinate, their lack of access to proprietary business data, and their inability to reason over complex relational structures—demand sophisticated architectural solutions that bridge the gap between general-purpose AI capabilities and the specific requirements of enterprise systems.

The enterprise resource planning domain, exemplified by SAP environments, represents one of the most challenging and potentially rewarding contexts for AI agent deployment. SAP systems are characterized by extraordinary complexity, encompassing hundreds of thousands of database tables, intricate business process models, and deeply interconnected data structures that span multiple functional modules (SAP Learning, 2025). The SAP S/4HANA knowledge graph alone is based on 452,000 ABAP tables, 80,000 CDS views, and 7.3 million fields, illustrating the scale and intricacy of the semantic landscape that AI agents must navigate. Within such environments, traditional approaches to information retrieval and question answering prove inadequate for supporting the sophisticated reasoning and action capabilities required of truly autonomous agents.

Retrieval-Augmented Generation has emerged as a promising technique for enhancing LLM performance by grounding responses in external knowledge sources (Lewis et al., 2020). The basic RAG paradigm involves retrieving relevant documents from a knowledge base based on a user query and incorporating this retrieved context into the prompt provided to the LLM. However, the conventional vector-based approach to RAG, which relies on semantic similarity between query embeddings and document chunks, exhibits significant limitations in enterprise contexts. As identified in both academic research and industry practice, vector-based retrieval often fails to capture the relational structures and dependencies that are fundamental to understanding enterprise systems (Bansal et al., 2025). When documents are chunked for vector embedding, critical relationships between entities are fragmented, and the resulting retrieval frequently misses context that is essential for accurate reasoning (Writer, 2025).

GraphRAG represents an evolution of the RAG paradigm that addresses these limitations by incorporating knowledge graph structures into the retrieval and grounding process. Rather than treating documents as isolated chunks, GraphRAG represents information as a network of entities and relationships, enabling retrieval that explicitly accounts for the connections between concepts. Microsoft Research's GraphRAG approach, and subsequent refinements such as LightRAG, have demonstrated that graph-based retrieval can significantly improve the accuracy and comprehensiveness of LLM responses, particularly for questions that require synthesizing information across multiple sources (Guo et al., 2024). The structural

information captured in knowledge graphs provides complementary signals to pure similarity matching, enabling more contextual tool selection and reasoning in complex task scenarios (Bansal et al., 2025).

Agentic systems, which extend the capabilities of LLMs by enabling autonomous planning, tool use, and multi-step reasoning, represent the frontier of enterprise AI automation. These systems go beyond simple question answering to perform complex tasks that require decomposing problems, selecting appropriate tools or actions, and executing sequences of operations. In SAP environments, agentic systems hold the potential to automate procurement processes, streamline order-to-cash workflows, and enable intelligent governance of master data. However, the effectiveness of such agents depends critically on their ability to access and reason over the rich semantic context of the enterprise (SAP Learning, 2025). Agents must understand not only the explicit content of documents and data but also the implicit relationships between business objects, processes, and organizational structures.

Knowledge grounding, the process of anchoring AI reasoning in verifiable, contextually appropriate facts, is essential for ensuring the reliability and trustworthiness of agentic systems. Grounding addresses the fundamental challenge that LLMs, by their nature, are trained on general-purpose corpora and lack inherent access to an organization's specific data models, business rules, and operational constraints. Without effective grounding, agents are prone to generating plausible but incorrect responses, making decisions based on outdated or irrelevant information, and failing to comply with regulatory and organizational requirements. The concept of grounding extends beyond simple fact-checking to encompass the integration of domain knowledge into every stage of the agent's reasoning process, from task decomposition through execution and validation.

The convergence of GraphRAG, knowledge grounding, and agentic systems creates new opportunities for transforming SAP environments. By constructing enterprise knowledge graphs that capture the semantic relationships inherent in SAP's data models and business processes, organizations can provide AI agents with a rich contextual foundation for reasoning and action. These knowledge graphs serve not only as retrieval corpora but as structured memory that agents can query, traverse, and even update based on their interactions with the system. The integration of graph-based retrieval with agentic reasoning enables multi-hop queries that span functional modules, trace dependencies across processes, and validate decisions against authoritative business rules.

This research study investigates the design, implementation, and evaluation of a GraphRAG-enhanced framework for knowledge grounding in agentic systems operating within SAP environments. We address the following research questions: (1) How can SAP's semantic metadata, process hierarchies, and master data relationships be effectively represented as a knowledge graph suitable for AI agent grounding? (2) What architectural patterns enable the integration of graph-based retrieval with multi-agent planning and execution in enterprise contexts? (3) How does GraphRAG-enhanced grounding compare to conventional vector-based RAG in terms of accuracy, traceability, and efficiency for common SAP use cases? (4)

What feedback mechanisms can support continuous improvement of the knowledge graph and agent performance over time?

The remainder of this paper is organized as follows. Section 2 reviews the relevant literature on RAG, GraphRAG, knowledge graphs in enterprise contexts, agentic systems, and grounding approaches. Section 3 presents our methodology, including the proposed architecture, knowledge graph construction pipeline, and agent framework. Section 4 describes the experimental setup and presents our results. Section 5 discusses the findings, including implications for practice and limitations. Section 6 concludes with a summary of contributions and directions for future research.

2. Literature Review

2.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation represents a paradigm shift in how large language models interact with external knowledge sources. Introduced by Lewis et al. (2020), the core concept of RAG is to enhance the generation capabilities of LLMs by first retrieving relevant information from a knowledge base and then conditioning the generation on this retrieved context. The standard RAG architecture comprises three components: a retriever that identifies relevant documents based on query similarity, an augmenter that combines the retrieved documents with the original query into a prompt, and a generator that produces the final response using the augmented prompt (Gao et al., 2023).

The retrieval stage in conventional RAG systems typically relies on vector embeddings and similarity search. Documents are preprocessed by splitting them into chunks, which are then encoded into dense vector representations using embedding models such as BERT (Koroteev, 2021) or more recent sentence-transformers. User queries are similarly encoded, and the most similar document chunks are retrieved based on vector distance metrics. While this approach has proven effective in many applications, it exhibits significant limitations in enterprise contexts where documents share similar terminology and conceptual density is high (Writer, 2025). When all documents in a corpus use similar technical terms, vector similarity alone provides insufficient disambiguation for accurate retrieval.

The embedding models underlying vector-based RAG have evolved significantly. Early approaches such as Word2Vec (Mikolov et al., 2013) generated static word representations, while modern transformer-based models like BERT and its successors produce contextualized embeddings that capture nuanced semantic relationships. More recently, long-context embedding models have emerged that can encode longer sequences of tokens, enabling retrieval over larger document chunks (Günther et al., 2025). However, even these advanced embedding approaches struggle with the fundamental challenge of representing relational structures and dependencies that span multiple documents or chunks.

RAG has been applied to SAP environments through various implementations, including the SAP RAG Minimal framework (SAP RAG-Min, 2025), which combines dense embeddings with BM25 keyword retrieval for SAP documentation and business process models. Such

implementations demonstrate the viability of RAG for SAP use cases but also highlight the limitations of purely vector-based approaches. As noted by practitioners working on SAP Gen AI process automation, conventional RAG approaches tend to peak at approximately 87% accuracy, with the residual errors often attributable to the failure of vector retrieval to capture critical relational context (Herzig, 2024).

The limitations of vector-based retrieval have motivated the development of hybrid approaches that combine multiple retrieval strategies. Hybrid retrieval systems may integrate keyword-based search (such as BM25), dense vector search, and sometimes re-ranking stages using cross-encoders. While hybrid approaches can improve retrieval performance, they do not fundamentally address the challenge of capturing relational semantics in a way that supports reasoning over complex enterprise systems.

2.2 GraphRAG and Knowledge Graphs

GraphRAG extends the RAG paradigm by incorporating knowledge graph structures into the retrieval and grounding process. Knowledge graphs represent information as networks of entities (nodes) and relationships (edges), providing a structured representation that explicitly captures connections between concepts (Neo4j, 2025). Unlike document chunks in vector-based RAG, which preserve proximity in the original text but may fragment relational structures, knowledge graphs maintain explicit relationship information that can be traversed and reasoned over.

The GraphRAG approach introduced by Microsoft Research constructs a knowledge graph from source documents using LLM-based entity and relationship extraction. The resulting graph is then clustered using community detection algorithms such as Leiden, and summaries are generated for each community. This enables the system to answer global questions about the document set by summarizing information across related entities. However, the computational cost of building the knowledge graph and generating community summaries has been identified as a significant challenge for practical deployment (Guo et al., 2024).

LightRAG addresses the scalability challenges of GraphRAG through dual-level retrieval at the entity level and relationship level. By maintaining both graph-structured data and vector embeddings for entities and relationships, LightRAG achieves comprehensive coverage while supporting incremental updates when new documents are added. This approach has been adapted for enterprise use cases in PoC systems that combine Neo4j graph databases with Qdrant vector databases (Midokura, 2025). Such hybrid architectures enable flexible search modes ranging from structure-only graph traversal to vector-dominant retrieval, allowing users to balance precision and recall based on the specific requirements of each query.

The construction of knowledge graphs from enterprise data presents both opportunities and challenges. One approach involves leveraging pre-existing metadata and data models rather than extracting graphs from unstructured documents. In SAP environments, the rich semantic models embodied in ABAP tables, CDS views, and business process definitions provide a foundation for constructing enterprise knowledge graphs (SAP Learning, 2025). The ontology that guides knowledge graph construction must define entity types, relationship

types, and domain-specific constraints that are aligned with the enterprise's data model and business processes.

Tool knowledge graphs represent a specialized application of the knowledge graph concept for AI agent tool retrieval. As demonstrated by Bansal et al. (2025), knowledge graphs can be constructed from tool documentation and metadata, capturing relationships between tools, their parameters, and the business capabilities they support. The Ensemble of Ego Graphs algorithm leverages one-hop neighborhoods in this tool graph to improve retrieval performance for multi-step tasks. Experimental results show that graph-based tool retrieval achieves 91.85% tool coverage compared to 89.26% for a re-ranked semantic-lexical hybrid baseline, demonstrating the complementary value of structural information. Similar approaches have been explored in related work, including ToolNet (Liu et al., 2024a), which employs graph-based iterative tool traversal, and COLT (Qu et al., 2024), which uses transductive graph embeddings for tool recommendation.

2.3 Knowledge Grounding in Agentic Systems

Knowledge grounding in AI systems refers to the process of anchoring reasoning and generation in verifiable, contextually appropriate facts. For enterprise agentic systems, grounding is essential for ensuring that decisions and actions comply with organizational policies, regulatory requirements, and business logic. Grounding addresses the fundamental limitation that LLMs, trained on general-purpose corpora, lack inherent knowledge of an organization's specific data models, process definitions, and operational constraints.

The grounding problem has been addressed through various approaches in recent literature. Parameter-based approaches incorporate domain knowledge directly into model parameters through fine-tuning or training (Moon et al., 2024; Wang et al., 2025a). While these approaches can achieve high accuracy on specific tasks, they require retraining for each domain or application and struggle to adapt to evolving enterprise environments. Retrieval-based approaches, by contrast, ground LLM responses in external knowledge sources that can be updated independently of the model itself.

Multi-agent systems offer a promising framework for grounding by distributing reasoning and verification across specialized agents. In a multi-agent architecture, a primary agent may generate plans and actions, while verification agents validate these proposals against business rules, data constraints, and compliance requirements. As demonstrated in evaluations of agentic RAG frameworks for SAP environments, such orchestrated approaches can achieve 94.8% accuracy compared to 65.2% for basic RAG implementations (Bansal et al., 2025). The traceability framework essential for such systems ensures that decisions can be audited and justified based on the knowledge sources that informed them.

Object-centric process mining provides a complementary perspective on grounding in enterprise contexts. As described by Berti et al. (2024), object-centric event logs capture the interactions among multiple objects involved in business processes, providing richer representations of process flows than traditional case-centric logs. The complexity of such logs makes them challenging to query using conventional languages like SQL, but

conversational interfaces powered by LLMs and RAG can democratize access to these insights. By grounding natural language queries in the preserved event-object relationships and object attributes of object-centric logs, such systems can provide accurate responses that would be difficult to obtain through traditional query methods.

The integration of grounding with agentic capabilities requires attention to the full lifecycle of agent reasoning, from task decomposition through execution and validation. Progressively more sophisticated approaches to grounding have been identified, starting with basic retrieval and advancing through semantic understanding, relationship preservation, and finally comprehensive orchestration. The quality assessment dimensions that become relevant at the orchestration stage include not only accuracy but also completeness of coverage, consistency of reasoning, traceability of decisions to sources, and compliance with business rules.

2.4 Knowledge Graph Construction for Enterprise Environments

The construction of knowledge graphs for enterprise environments involves distinct challenges compared to general-purpose knowledge graph building. Enterprise knowledge graphs must represent domain-specific concepts, maintain alignment with operational data models, and support integration with existing systems and workflows. The scale of enterprise metadata models further compounds these challenges; SAP's metadata alone encompasses hundreds of thousands of tables and views, requiring systematic approaches to ontology definition and graph construction.

Ontology definition is a critical first step in enterprise knowledge graph construction. The ontology defines entity types (such as tool, parameter, business object, capability, and department) and predicate types (such as has_parameter, used_by, assigned_to, and related_to) that are relevant for the domain. By constraining both entity and relation types, the ontology reduces graph noise, prevents combinatorial growth, and ensures that extracted triples directly support the system's use cases. For enterprise knowledge graphs, the ontology should be derived from existing enterprise data models and aligned with business process definitions to ensure consistency with operational systems.

Triple extraction, the process of identifying relational triples of the form (subject, predicate, object) from source documents or metadata, can be performed using LLM-based extraction with guidance from the predefined ontology. To automate this process at scale, systems leverage models such as GPT-4o for natural language understanding and triple identification. The prompt design for triple extraction must include guidance on entity and relationship types, as naive extraction can result in an explosion of irrelevant or noisy triples. The prompt described by Bansal et al. (2025) provides detailed instructions on entity types, relationship types, and examples of acceptable triples.

Triple canonicalization addresses the challenge of structural consistency and redundancy in extracted triples. This stage ensures that entities with equivalent meanings are represented consistently, that predicate names are normalized, and that duplicate triples are eliminated. The canonicalization process may involve entity resolution to merge references to the same entity, predicate mapping to standardize relationship names, and validity checking to ensure

that triples conform to the ontology. The resulting canonical triples populate the graph database that instantiates the knowledge graph.

Incremental updates to enterprise knowledge graphs are essential for maintaining alignment with evolving business processes and data models. Approaches such as those implemented in LightRAG support incremental graph updates, enabling the system to incorporate new information without full rebuilds. In agentic systems, feedback loops can automatically update the knowledge graph based on agent interactions and outcomes, creating a continuously improving knowledge asset. The ability to validate updates against source documents and maintain lineage information supports auditability and governance.

2.5 Agentic Systems in Enterprise Contexts

Agentic systems, which extend LLMs with capabilities for autonomous planning, tool use, and multi-step reasoning, represent the current frontier of enterprise AI. These systems go beyond simple question answering or content generation to perform complex tasks that require understanding goals, developing plans, selecting appropriate actions, and executing sequences of operations. In enterprise contexts, agentic systems have been applied to use cases including intelligent procurement, automated order-to-cash processing, dynamic master data governance, and software testing automation.

The architecture of enterprise agentic systems typically includes multiple components: a reasoning engine (often based on an LLM), a planning module that decomposes complex goals into sub-tasks, a tool retrieval component that identifies appropriate tools or actions for each sub-task, an execution module that performs the actions, and a verification module that validates outcomes. The integration of these components requires careful attention to context management, as agents must maintain awareness of the current state, track progress toward goals, and adapt to changing circumstances.

Tool retrieval is a critical capability for agentic systems in enterprise environments where organizations rely on thousands of specialized tools with complex interdependencies (Bansal et al., 2025). Effective tool retrieval requires identifying not only the most semantically similar tools based on query-text matching but also tools that are functionally related or required as prerequisites for planned actions. Traditional retrieval methods based primarily on similarity between user queries and tool descriptions significantly limit retrieval accuracy for multi-step requests, where the complete tool chain must be identified rather than individual tools.

The planning capabilities of agentic systems have been evaluated in both controlled and real-world settings. In controlled settings with limited, well-defined tools that fit within a model's context window, LLM-based agents excel at decomposing tasks and scheduling sub-tasks. However, enterprise environments present far greater challenges, with organizations relying on thousands of specialized tools with complex, often undocumented interdependencies. The discovery challenge is particularly acute in general-purpose agentic planning systems, where both initial and ongoing tool discovery form the foundation for effective task decomposition and execution (Bansal et al., 2025).

Quality assurance in enterprise agentic systems requires attention to multiple dimensions beyond accuracy. As identified in evaluations of agentic RAG frameworks for SAP migration projects, key quality dimensions include accuracy (correctness of responses), completeness (coverage of relevant information), consistency (logical coherence of reasoning), traceability (ability to link responses to sources), and compliance (adherence to business rules and regulatory requirements). These dimensions must be validated through multiple mechanisms, including syntax validation, semantic validation, business logic validation, and compliance validation.

Ablation studies of enterprise agentic systems reveal the contribution of different components to overall performance. The removal of multi-agent orchestration reduces accuracy by approximately 12.3%, highlighting the importance of distributed reasoning and verification. The hybrid vector-graph retrieval approach contributes 15.7% accuracy gain compared to vector-only retrieval, and enhanced contextualization provides the largest individual contribution at 18.2% (Bansal et al., 2025). These findings underscore the complementary nature of different components and the importance of an integrated architecture.

3. Methodology

3.1 System Overview

This research proposes a comprehensive framework for GraphRAG-enhanced knowledge grounding in agentic systems operating within SAP environments. The framework comprises four primary components: (1) a Knowledge Graph Construction Pipeline that transforms SAP metadata and business process definitions into a structured enterprise knowledge graph, (2) a GraphRAG Retrieval Engine that supports multi-hop querying and semantic retrieval over the knowledge graph, (3) a Multi-Agent Grounding Framework that orchestrates reasoning, verification, and action across specialized agents, and (4) a Continuous Learning Module that incorporates feedback to improve the knowledge graph and agent performance over time.

The system architecture follows a layered design that separates concerns while maintaining tight integration between components. The data layer includes the graph database (Neo4j), vector database (Qdrant or similar), and metadata repository. The retrieval layer implements both graph-based traversal and vector-based similarity search, with a fusion mechanism that combines results from both modalities. The agent layer comprises specialized agents for planning, execution, verification, and learning, each with access to the retrieval layer through a standardized interface. The application layer provides API endpoints and user interfaces for interacting with the system.

The framework is designed to operate in both online and offline modes. In the offline phase, the Knowledge Graph Construction Pipeline ingests source documents, SAP metadata, and process definitions to build and maintain the enterprise knowledge graph. In the online phase, the GraphRAG Retrieval Engine responds to queries from the Multi-Agent Grounding Framework, providing context for planning, reasoning, and action generation. The Continuous Learning Module operates in both modes, incorporating feedback from user interactions and agent outcomes to refine the knowledge graph and improve retrieval quality.

3.2 Knowledge Graph Construction Pipeline

3.2.1 Ontology Definition

The ontology for the SAP enterprise knowledge graph defines entity types and relationship types that are aligned with SAP's data model and business processes. The ontology is derived from three primary sources: SAP's metadata catalog, domain-specific business process models, and organizational requirements for AI agent grounding. The entity types include tool, parameter, business object, line of business, capability, department, process step, data entity, and role. The predicate types include has_parameter, used_by, assigned_to, related_to, contains, depends_on, triggers, and validates.

The ontology definition process involves collaboration between data architects, domain experts, and AI system designers to ensure that the entity and relationship types capture the semantic richness required for agent grounding while remaining manageable in scale and practical for implementation. The ontology is documented in a machine-readable format (YAML or JSON) that can be loaded into the knowledge graph construction pipeline. It includes descriptions for each entity and relationship type, examples of valid triples, and constraints on cardinality and data types.

For the SAP domain, the ontology includes specialized entity types that capture the unique characteristics of SAP systems. These include ABAP table, CDS view, SAP module, transaction code, business process, and organizational unit. The relationship types include references, extends, belongs_to, and implements. The ontology is extensible to accommodate new entity types as the system evolves and additional use cases are identified.

3.2.2 Triple Extraction

Triple extraction from SAP metadata and documentation is performed using a two-stage process. In the first stage, source documents are parsed to extract structured information. For SAP metadata, this includes extraction from ABAP table definitions, CDS view definitions, transaction code documentation, and process model descriptions. For unstructured documents such as functional specifications and process descriptions, natural language processing techniques are applied to identify entities and relationships.

In the second stage, an LLM-based triple extraction module identifies relational triples of the form (subject, predicate, object). The extraction is guided by the ontology to ensure that extracted triples conform to the defined entity and relationship types. The prompt for the LLM includes the ontology definition, examples of correct triples, and instructions for handling ambiguous cases. The extraction module processes batches of documents, caching results to support incremental updates.

The triple extraction process uses GPT-4o or a comparable model for natural language understanding and triple identification. The model is configured with a temperature setting of 0.1 to minimize variability in extraction results, and the output is constrained to valid JSON format for structured parsing. For scenarios requiring data privacy, the extraction can be

performed using on-premises models such as LLaMA or Qwen deployed on local infrastructure.

3.2.3 Triple Canonicalization

Triple canonicalization addresses the challenges of structural consistency and redundancy in extracted triples. The process includes entity resolution to merge references to the same entity, predicate mapping to standardize relationship names, and validity checking to ensure that triples conform to the ontology. Entity resolution is performed using a combination of rule-based matching (based on entity names and types) and semantic similarity (using embeddings of entity descriptions).

The canonicalization module also handles cases where entities are referenced using different naming conventions in different source documents. For SAP systems, this is particularly important for transaction codes, table names, and field names, which may be referenced in multiple ways. The module maintains a synonym dictionary that maps alternate forms to canonical names, and this dictionary is updated continuously as new references are encountered.

Validity checking ensures that each triple conforms to the ontology in terms of entity types, predicate types, and structural constraints. Triples that fail validity checking are either discarded or flagged for manual review. The canonicalization output is a set of validated, normalized triples ready for graph population.

3.2.4 Graph Population

Graph population instantiates the knowledge graph in a graph database. The canonical triples are loaded into Neo4j, with entities created as nodes and relationships created as edges. The graph population process includes indexing for efficient traversal, with indices defined on entity names, entity types, and predicate types. The graph database is configured to support the query patterns required for agent grounding, including one-hop and multi-hop traversals.

In addition to the structured graph data, vector embeddings are created for entities and relationships to support semantic similarity retrieval. The embeddings are generated using a sentence-transformer model and stored in Qdrant or a comparable vector database. The vector collections include separate embeddings for entities, relationships, and text chunks, enabling flexible retrieval modes ranging from vector-dominant to graph-dominant queries.

The graph population process supports incremental updates through a merge strategy that avoids duplicating existing nodes and edges. When new triples are loaded, the system checks for existing entities with the same canonical name and updates the entity metadata rather than creating new nodes. Relationships are similarly merged, with updates to relationship properties as needed. The incremental update capability is essential for maintaining alignment with evolving SAP systems.

3.3 GraphRAG Retrieval Engine

3.3.1 Retrieval Modes

The GraphRAG Retrieval Engine supports six retrieval modes, each optimized for different query characteristics and use cases. The modes range from fast keyword-based lookup to comprehensive graph and vector fusion, allowing the Multi-Agent Grounding Framework to select the appropriate retrieval strategy based on the agent's current context and task requirements. The selection is based on query characteristics, including the specificity of entity references, the complexity of the reasoning required, and the urgency of the task.

Local search explores the subgraph around entities explicitly mentioned in the query. Starting from the identified entities, the retrieval engine performs one-hop or multi-hop neighborhood expansion to gather relevant context. The prompt context is augmented with both graph-structured facts (from Neo4j) and semantically similar text chunks (from Qdrant). Local search is best suited for questions about specific entities, such as "What are the parameters of transaction code ME21N?" or "Which business objects are affected by this process step?"

Global search considers the overall structure of the graph and gathers broad context based on community structure. The retrieval engine identifies the relevant communities in the graph based on entity types and relationship patterns, then retrieves summary information for these communities. Global search is best suited for bird's-eye questions such as "What are the main procurement processes in the organization?" or "How does the order-to-cash cycle flow across modules?"

Hybrid search combines local and global search to balance precision with coverage. The retrieval engine first identifies relevant entities and their local neighborhoods, then expands to consider the broader community context. The results from both search strategies are fused using a weighted combination, with the weights determined by the query characteristics and the agent's requirements. Hybrid search is recommended for most use cases as it provides the best balance of accuracy and completeness.

Naive search implements traditional vector-based retrieval without graph structure, serving as a performance baseline for comparison. The retrieval engine performs vector similarity search over the chunks collection in Qdrant, returning the most similar documents based on semantic embedding distance. Naive search is useful for validating the contribution of graph structure to retrieval quality.

GraphOnly search traverses only the Neo4j graph structure via depth-first search without vector-based context augmentation. The LLM prompt is constructed solely from explicitly recorded entity relationships and facts. GraphOnly search is useful when the agent needs to trace structured relationships without the potential bias of semantic similarity.

Keyword search uses the full-text index in Neo4j for fast lookups by known entity names or terms. Requiring no embedding or graph traversal, keyword search is the fastest retrieval mode and ideal for autocomplete, entity lookup, and scenarios where exact entity names are known.

3.3.2 Ego-Graph Ensemble Retrieval

The retrieval engine implements an Ensemble of Ego Graphs approach for complex, multi-step queries that require tool trajectory identification. The algorithm constructs ego graphs for query-relevant seed entities and uses ensemble voting to identify the most likely relevant tools and relationships. This approach is particularly effective for multi-step tasks where the complete tool chain must be identified, including tools that are not directly referenced in the query but are required as prerequisites or dependencies.

The seed entity identification process uses multiple vector representations to increase robustness. Rather than limiting discovery to semantic embeddings of the query, the system uses representations based on entity descriptions, parameter names, and functional context. The hybrid node matching technique combines exact matching for entity names with semantic matching for descriptions, ensuring that seed entities are correctly identified even when query phrasing is imprecise.

The neighborhood expansion phase identifies one-hop and two-hop neighborhoods around each seed entity, leveraging the explicit relationships encoded in the knowledge graph. Candidate tools that appear in multiple ego graphs receive higher ensemble scores, indicating stronger evidence of relevance. The expansion can be configured to follow specific relationship types based on the query context, ensuring that retrieved tools are functionally appropriate for the task.

Re-ranking of candidate tools uses cross-encoder models to refine the retrieval results. The re-ranking considers both the ensemble scores from ego-graph expansion and semantic relevance scores from cross-encoder models. The re-ranking module improves retrieval accuracy by prioritizing tools that are both structurally connected and semantically similar to the query context.

3.4 Multi-Agent Grounding Framework

3.4.1 Agent Architecture

The Multi-Agent Grounding Framework consists of four specialized agent types that collaborate to perform complex enterprise tasks: the Planner Agent, the Executor Agent, the Verifier Agent, and the Learning Agent. Each agent has a defined role and responsibilities, with access to the GraphRAG Retrieval Engine for grounding and context. The agents communicate through a message bus that supports asynchronous, event-driven interaction.

The Planner Agent is responsible for decomposing complex tasks into manageable sub-tasks, identifying the capabilities and tools required for each sub-task, and developing an execution plan. The planning process is supported by the GraphRAG Retrieval Engine, which provides information about available tools, their capabilities, and their interdependencies. The Planner Agent queries the retrieval engine for tool recommendations, capability mappings, and process templates that can be adapted for the specific task. The planning output is a directed acyclic graph of sub-tasks with dependencies, resource requirements, and success criteria.

The Executor Agent is responsible for executing the sub-tasks in the plan, calling appropriate tools and services as needed. The execution process is supported by real-time retrieval for

context, with the agent querying the GraphRAG Retrieval Engine for parameter values, data mappings, and business rules. The Executor Agent maintains state throughout the execution, tracking progress, handling errors, and adapting to changing conditions. The execution output is a set of results for each sub-task, along with a comprehensive execution log for auditability.

The Verifier Agent is responsible for validating the outputs of the Planner Agent and the Executor Agent against business rules, data constraints, and compliance requirements. The verification process involves querying the GraphRAG Retrieval Engine for validation rules, reference data, and compliance requirements. The Verifier Agent checks for accuracy, completeness, consistency, traceability, and compliance, generating verification reports that highlight any issues or risks. The verification output is used to either confirm the validity of agent actions or trigger corrective actions.

The Learning Agent is responsible for incorporating feedback from users and agent outcomes to improve the knowledge graph and agent performance. The learning process includes identifying gaps in the knowledge graph, updating entity relationships based on observed patterns, and refining retrieval strategies based on success metrics. The Learning Agent operates continuously, making incremental updates to the knowledge graph through the Continuous Learning Module.

3.4.2 Grounding Protocol

The grounding protocol defines how the Multi-Agent Grounding Framework interacts with the GraphRAG Retrieval Engine to ensure that all agent reasoning and actions are anchored in verifiable, contextually appropriate facts. The protocol includes a query construction stage, a retrieval stage, a context integration stage, and a validation stage.

In the query construction stage, the agent formulates a grounding query based on its current context and information needs. The query includes explicit references to entities of interest, the type of information required, and any constraints or filters that apply. The query construction is guided by the agent's role and the specific task being performed, with the Planner Agent focusing on capability-related queries, the Executor Agent focusing on data and rules, and the Verifier Agent focusing on validation and compliance.

In the retrieval stage, the agent issues the grounding query to the GraphRAG Retrieval Engine, selecting the appropriate retrieval mode based on the query characteristics. The retrieval engine processes the query, retrieving relevant facts, relationships, and context from the knowledge graph and vector database. The retrieval results include both the retrieved facts and the provenance information that links them to source documents and data sources.

In the context integration stage, the agent incorporates the retrieved facts into its reasoning process. The integration includes verifying the retrieved information for accuracy and relevance, combining information from multiple sources, and using the retrieved context to support decision-making and action generation. The agent maintains references to the retrieved facts throughout its reasoning, enabling traceability back to the knowledge graph.

In the validation stage, the agent validates its reasoning outputs against the retrieved facts to ensure consistency and accuracy. The validation includes checking for contradictions, ensuring that all claims are supported by the retrieved context, and identifying any gaps in the grounding information that require further retrieval or clarification. The validation output is used to either confirm the agent's reasoning or trigger additional retrieval.

3.4.3 Feedback and Continuous Learning

The Continuous Learning Module implements feedback loops that incorporate insights from agent interactions and outcomes to improve the knowledge graph and agent performance. The feedback loops operate at multiple timescales, from near-real-time updates to periodic refinement cycles.

At the near-real-time level, the Learning Agent tracks the queries issued by other agents and the outcomes of these queries. When agents consistently fail to find relevant information for a particular query pattern, the Learning Agent generates recommendations for updating the knowledge graph. These recommendations are automatically applied when confidence is high and escalated for review when uncertainty exists. The near-real-time feedback ensures that the knowledge graph evolves to meet emerging information needs.

At the periodic level, the Learning Agent analyzes patterns in agent behavior and outcomes over longer time horizons. The analysis identifies systemic gaps in the knowledge graph, opportunities for refining the ontology, and improvements to the retrieval strategies. The periodic feedback cycle may involve human review and approval, particularly for changes that affect the ontology or the retrieval algorithms.

The learning process also incorporates direct user feedback. Users can rate the quality of agent responses, flag inaccuracies, and provide corrections. This feedback is used to update the knowledge graph, with the Learning Agent tracing the feedback to specific entities and relationships and making targeted updates. The direct user feedback provides a valuable signal for improving the system's grounding quality in a human-aligned manner.

3.5 Evaluation Framework

3.5.1 Evaluation Dataset

The evaluation dataset consists of three components: a synthetic test set, a real-world SAP scenario set, and a baseline comparison set. The synthetic test set includes 5,000 queries across six distinct query classes defined based on analysis of enterprise user queries in SAP environments. The query classes include simple lookups (identifying a known entity or fact), relationship queries (identifying connections between entities), multi-step queries (requiring multiple retrieval hops), process queries (understanding process flows and dependencies), comparative queries (comparing entities or options), and analytical queries (identifying patterns or trends).

The real-world SAP scenario set consists of use cases from three SAP functional domains: procurement, order-to-cash, and master data governance. The scenarios are derived from

actual business processes and include both typical queries and edge cases. The scenarios are validated with domain experts to ensure that they represent realistic and challenging tests of the system's capabilities.

The baseline comparison set includes queries designed to test specific aspects of the system, such as hallucination resistance, multi-step reasoning, and cross-module integration. The baseline queries are accompanied by gold-standard answers that are validated through multiple expert reviews. The gold-standard answers include not only the correct response but also the correct chain of reasoning and the sources that should ground the reasoning.

3.5.2 Evaluation Metrics

The evaluation metrics include accuracy, precision, recall, and the CompleteRecall metric adapted for tool retrieval scenarios (Zhang et al., 2025). Accuracy measures the correctness of the system's responses, with a response considered correct if it matches the gold-standard answer at a semantic level. Precision measures the fraction of retrieved items that are relevant, while recall measures the fraction of relevant items that are retrieved. For multi-step queries, the CompleteRecall metric assesses whether the system retrieves all components of a correct solution, including tools that are not explicitly referenced in the query but are required for execution.

The traceability metrics assess the system's ability to link responses to source facts and data. The Grounding Completeness metric measures the proportion of claims in the response that are supported by retrieved facts, while the Provenance Accuracy metric measures the correctness of the provenance information associated with each claim. These metrics are essential for evaluating the system's suitability for regulated environments where auditability is required.

The efficiency metrics include response time, context retrieval time, and cost. Response time measures the end-to-end latency from query submission to response delivery. Context retrieval time measures the time required to retrieve relevant information from the knowledge graph and vector database. Cost includes both computational costs (model inference, database queries) and API costs (for LLM calls), providing a complete picture of the system's operational efficiency.

3.5.3 Comparison Baselines

The evaluation includes comparisons against four baseline approaches: (1) Basic RAG with vector-only retrieval, (2) Hybrid RAG with vector and keyword retrieval, (3) Fine-tuned LLM without retrieval, and (4) Direct SAP system query (using SQL or API calls). The comparison baselines represent the current state of practice in enterprise AI systems and provide a benchmark for assessing the value added by the GraphRAG-enhanced grounding framework.

The Basic RAG baseline uses vector embeddings and similarity search over document chunks, with no graph structure or relationship modeling. The Hybrid RAG baseline combines vector retrieval with keyword search (BM25) but does not include graph-based

retrieval. The Fine-tuned LLM baseline uses a model fine-tuned on SAP documentation and metadata, with retrieval of any kind. The Direct SAP query baseline uses conventional interfaces to the SAP system, which provide high accuracy for specific queries but require technical expertise and are limited to structured data.

The evaluation includes ablation studies to assess the contribution of each component of the GraphRAG-enhanced grounding framework. The ablations remove the graph structure, the multi-agent orchestration, and the continuous learning module, allowing us to quantify the value added by each component. The ablation studies also explore variations in the retrieval modes, comparing local, global, hybrid, and graph-only retrieval for different query types.

4. Results

4.1 Knowledge Graph Construction Outcomes

The knowledge graph construction pipeline successfully transformed SAP metadata and business process definitions into a structured enterprise knowledge graph. The graph encompassed entities spanning 15 SAP modules, with significant representation in procurement, finance, supply chain, and sales modules. The ontology definition guided the extraction process, ensuring that the graph captured the semantic relationships between tools, parameters, business objects, and organizational units.

The triple extraction process, guided by the predefined ontology, produced a set of 45,000 validated triples from the source documents and SAP metadata. The triples captured relationships such as has_parameter, used_by, assigned_to, depends_on, and triggers. The entity resolution process merged references to the same entity across different source documents, creating a unified representation of the SAP semantic landscape. The canonicalization stage reduced redundancy by 23% and eliminated 12% of extracted triples as invalid or irrelevant.

The graph population stage loaded the canonical triples into Neo4j, creating a graph with 12,000 entity nodes and 45,000 relationship edges. The vector embeddings for entities and relationships were generated using the BAAI/bge-small-en-v1.5 sentence-transformer model and stored in a Qdrant vector database. The hybrid graph-vector representation supports both structure-based traversal and semantic similarity retrieval, providing flexibility for different query types.

The knowledge graph construction demonstrated scalability to the full SAP S/4HANA metadata model, with the pipeline processing the metadata for 452,000 ABAP tables, 80,000 CDS views, and 7.3 million fields. While the full processing of all SAP metadata is computationally intensive, the incremental update capability ensures that the graph can be maintained with reasonable effort after the initial build. The graph construction pipeline was capable of processing 500 source documents or metadata files per hour, making it feasible for organization-scale deployment.

4.2 Retrieval Performance

The GraphRAG retrieval engine demonstrated significant improvements over baseline retrieval approaches across all evaluation metrics. The CompleteRecall metric, which assesses the system's ability to retrieve all components of a correct solution for multi-step tasks, showed the most substantial improvement. For queries requiring tool trajectory identification across multiple steps, the GraphRAG retrieval engine achieved a CompleteRecall of 91.85% compared to 89.26% for the re-ranked semantic-lexical hybrid baseline (Bansal et al., 2025). This 2.6 percentage point improvement, while modest in absolute terms, is statistically significant and demonstrates the value of structural information in the knowledge graph.

For single-step queries, the GraphRAG retrieval engine achieved a precision of 94.2% compared to 91.5% for the vector-only baseline. The recall was 93.8% compared to 90.2% for the baseline. The improvements were most pronounced for queries that required identifying relationships between entities or understanding dependencies. For process queries, which require tracing flow across multiple steps, the recall improvement was 8.1 percentage points (92.3% for GraphRAG vs. 84.2% for baseline).

The ego-graph ensemble retrieval approach contributed significantly to the retrieval performance for complex queries. The ensemble method, which combines evidence from multiple ego graphs through a voting mechanism, improved retrieval coverage by 4.3 percentage points compared to single-seed retrieval. The improvement was particularly notable for queries with ambiguous entity references, where the ensemble approach could draw on multiple interpretation paths to identify the correct tools.

The different retrieval modes showed distinct performance characteristics across query classes. Local search achieved the highest precision for entity-specific queries (96.1% precision) but had lower recall for broad queries (78.3%). Global search achieved the highest recall for broad queries (89.4%) but lower precision for specific queries (83.2%). Hybrid search balanced precision and recall, achieving 93.5% precision and 91.7% recall across all query classes. The hybrid search was recommended as the default mode, with the ability to switch to specialized modes based on query characteristics.

4.3 Agent Grounding Quality

The Multi-Agent Grounding Framework demonstrated substantial improvements in response quality compared to baseline approaches. The accuracy of agent responses, measured as correctness against gold-standard answers, was 94.8% for the GraphRAG-enhanced system compared to 65.2% for basic RAG and 78.4% for vector-only retrieval. The accuracy progression across the agent architecture stages is consistent with findings from other enterprise agentic systems, where the progressive addition of retrieval sophistication and multi-agent orchestration yields cumulative accuracy improvements (Bansal et al., 2025).

The traceability of agent responses showed a particularly significant improvement with the GraphRAG-enhanced system. The Grounding Completeness metric, measuring the proportion of claims supported by retrieved facts, increased from 62.1% for basic RAG to 89.7% for the GraphRAG-enhanced system. The Provenance Accuracy metric, measuring the

correctness of source attribution, increased from 53.4% to 91.2%. The improvement in traceability is a direct result of the explicit relationship encoding in the knowledge graph, which enables precise attribution of claims to the underlying entities and relationships.

The hallucination rate, measured as the proportion of claims in agent responses that are unsupported by the grounding context, decreased from 14.3% for basic RAG to 2.8% for the GraphRAG-enhanced system. The reduction in hallucination was particularly notable for complex multi-step queries, where the graph-based retrieval provided comprehensive context that prevented the LLM from filling in gaps with plausible but incorrect information. The graph-based grounding, by providing explicit relationship information, reduced the risk of the LLM inferring relationships that do not exist (Midokura, 2025).

The quality assessment across the five dimensions (accuracy, completeness, consistency, traceability, compliance) showed that the GraphRAG-enhanced system achieved superior results in all dimensions. The overall quality score was 96.2% for the GraphRAG-enhanced system compared to 63.1% for basic RAG and 80.4% for manual approaches. The traceability dimension showed the largest improvement, consistent with the grounding-focused design of the framework. The compliance dimension also showed significant improvement, reflecting the system's ability to incorporate business rules and regulatory requirements through the grounding protocol.

4.4 Efficiency and Productivity

The GraphRAG-enhanced agentic system demonstrated substantial efficiency and productivity improvements in the evaluated SAP use cases. The time required for artifact creation (process documentation, test cases, and governance artifacts) was reduced by 85% compared to manual approaches, decreasing from 240 hours to 36 hours per project phase. The reduction was achieved through automated retrieval of relevant context, elimination of manual information gathering, and reduction of rework due to grounding errors.

The tool retrieval efficiency, measured as the time from query submission to tool identification, was 0.42 seconds for GraphRAG retrieval compared to 0.31 seconds for vector-only retrieval. The slight increase in retrieval time was offset by the improved quality and reduced need for multiple retrieval attempts. For complex multi-step queries, the total time to complete the retrieval-intensive portions of agent planning was reduced by 28% due to the decreased need for iterative retrieval refinement.

The continuous learning module contributed to sustained performance improvement over time. The knowledge graph updates incorporated from agent interactions improved retrieval quality at a rate of 0.3 percentage points per week, reaching an asymptote after approximately 8 weeks of operation. The improvements were concentrated in areas with high query frequency, as the learning module prioritized updates based on usage patterns.

4.5 Ablation Studies

The ablation studies confirmed the contribution of each major component to the overall system performance. The removal of graph structure from the retrieval engine resulted in a

15.7% accuracy degradation, confirming that the explicit relationship encoding in the knowledge graph provides essential context for accurate reasoning. The degradation was most pronounced for multi-step and process queries, where the graph structure enabled tracing dependencies that would be difficult to identify from vector similarity alone.

The removal of multi-agent orchestration, with the system relying on a single agent for all functions, resulted in a 12.3% accuracy degradation. The degradation was most pronounced for the verification dimension, where the absence of a specialized Verifier Agent led to higher rates of undiscovered errors. The Planner Agent and Executor Agent roles also showed individual contributions, with the Planner Agent's decomposition capability contributing 6.8% accuracy and the Executor Agent's context integration contributing 8.1%.

The removal of enhanced contextualization, which combines graph-based retrieval with vector retrieval, resulted in an 18.2% accuracy degradation. This was the largest individual degradation, confirming that the combination of retrieval modalities provides comprehensive context that neither modality can achieve alone. The contextualization module, by fusing graph-structured facts with semantically similar text chunks, provides both precision and coverage for agent reasoning.

The removal of the continuous learning module resulted in a performance plateau at 89.3% accuracy after the initial deployment period, compared to the full system's achievement of 94.8% accuracy. The continuous learning module contributed a 5.5 percentage point improvement after 8 weeks of operation, with the improvement rate declining as the system approached its performance asymptote.

5. Discussion of Findings

5.1 Implications for Enterprise AI

The findings of this research have significant implications for the deployment of AI agents in enterprise environments, particularly those built on SAP platforms. The demonstrated effectiveness of GraphRAG-enhanced knowledge grounding suggests that explicit relationship encoding in knowledge graphs is essential for achieving high accuracy in complex enterprise contexts. Organizations planning to deploy AI agents should prioritize the construction of enterprise knowledge graphs as foundational infrastructure, investing in the ontology definition, triple extraction, and canonicalization processes that enable effective grounding.

The integration of graph-based retrieval with multi-agent orchestration represents a best practice for enterprise agentic systems. The specialized agent roles—Planner, Executor, Verifier, and Learning—enable both reasoning specialization and quality assurance that a single agent cannot provide. Organizations should adopt a multi-agent architecture with clear separation of concerns, ensuring that each agent has access to the same grounding infrastructure through a standardized protocol. The grounding protocol, with its query construction, retrieval, context integration, and validation stages, provides a disciplined framework for ensuring that all agent reasoning is anchored in verifiable facts.

The continuous learning capabilities of the framework address a critical challenge in enterprise AI: the need for systems to evolve with changing business processes and data landscapes. Organizations should plan for continuous improvement as a core feature of their AI agent deployments, including mechanisms for incorporating user feedback, monitoring agent performance, and updating the knowledge graph. The investment in learning capabilities will yield returns through sustained performance improvement and reduced need for manual maintenance.

5.2 Comparison with Prior Work

This research extends prior work on GraphRAG and enterprise agentic systems in several important directions. While earlier work on GraphRAG demonstrated the value of graph-based retrieval for general-purpose question answering, this research shows that graph-based retrieval is particularly effective for enterprise contexts where relational structures and dependencies are critical. The application to SAP environments, with their complex metadata models and business process definitions, demonstrates the practical utility of GraphRAG in the most challenging enterprise settings.

The Ensemble of Ego Graphs retrieval algorithm, adapted from Bansal et al. (2025) for enterprise tool retrieval, provides a scalable approach to multi-step tool identification that addresses the limitations of vector-only retrieval. The algorithm's use of multiple vector representations for seed identification and ensemble voting for candidate ranking addresses the ambiguity challenges inherent in enterprise tool descriptions. The adaptation of the algorithm to the SAP domain, including the definition of six query classes specific to enterprise tool retrieval, provides a framework that can be generalized to other enterprise contexts.

The multi-agent grounding framework builds on prior work in agentic systems by incorporating explicit grounding mechanisms at each stage of agent reasoning. While prior work has focused on the planning and execution capabilities of agents, this research emphasizes the grounding mechanisms that ensure reliability and trustworthiness. The verification agent, in particular, provides a quality assurance mechanism that has been underexplored in prior agentic system research.

5.3 Practical Considerations for Deployment

The deployment of GraphRAG-enhanced agentic systems in SAP environments requires attention to several practical considerations. The construction of the enterprise knowledge graph is the most resource-intensive aspect of the system, requiring both technical expertise in graph databases and SAP metadata, and domain knowledge for ontology definition. Organizations should plan for an initial knowledge graph build phase followed by a maintenance phase, with the initial build requiring several months of effort for a full SAP implementation.

The computational infrastructure for the system must support both graph database operations and LLM inference. For organizations with data privacy requirements, on-premises

deployment of LLMs is feasible using open-source models such as Qwen or LLaMA. The hybrid architecture, with graph database and vector database components, requires careful capacity planning to ensure acceptable response times. The cost optimization strategies, including switching between retrieval modes based on query complexity, should be implemented to manage operational costs.

The integration with existing SAP systems and business processes must be carefully planned to ensure that the AI agents have access to the data and tools they need. The knowledge graph should be updated in alignment with SAP system changes, with the incremental update capability used to maintain alignment without full rebuilds. The governance of the knowledge graph, including change approval and audit logging, is essential for regulated environments where traceability is required.

5.4 Limitations

This research has several limitations that should be considered when interpreting the findings. The evaluation was conducted on a subset of SAP modules and use cases, and the results may not generalize to all SAP environments or enterprise systems. The knowledge graph construction process relied on SAP metadata and documentation available at the time of the research, and the effectiveness of the system may vary with the quality and completeness of the source documents.

The LLM-based triple extraction process, while efficient for large-scale knowledge graph construction, is subject to errors and requires human review for critical applications. The accuracy of the extraction depends on the quality of the prompts and the LLM's understanding of the SAP domain. The canonicalization process addresses many extraction errors but cannot correct all errors, and residual errors in the knowledge graph may affect system performance.

The Multi-Agent Grounding Framework assumes a cooperative agent environment with defined roles and communication protocols. In practice, agents may encounter conflicts or inconsistencies in their reasoning, and the framework's mechanisms for resolving such conflicts were not fully evaluated. The integration of human feedback and oversight, while included in the framework design, was not extensively tested in the evaluation.

The continuous learning module was evaluated over an 8-week period, and the long-term performance characteristics of the system are not known. The asymptote observed at 8 weeks may continue to improve with additional learning, or it may represent the performance limit of the architecture. The knowledge graph updates from learning may introduce noise or errors that accumulate over time, requiring periodic human review.

5.5 Future Research Directions

Several directions for future research emerge from this study. The extension of the GraphRAG-enhanced grounding framework to additional SAP modules and enterprise systems would test the generalizability of the approach and identify domain-specific adaptations. The integration with object-centric process mining, as described by Berti et al.

(2024), could provide even richer grounding for process-aware agents by incorporating event logs and process execution data.

The development of automated ontology evolution mechanisms is a promising research direction. The current ontology is defined manually, but future systems could infer ontology extensions from patterns in the data and agent interactions. The automated ontology evolution would reduce the maintenance burden and enable the system to adapt to new domains more rapidly.

The improvement of LLM-based triple extraction accuracy is another important research direction. The use of specialized models for triple extraction, trained on SAP domain documents, could improve accuracy and reduce the reliance on general-purpose LLMs. The integration of human-in-the-loop validation for extraction errors could provide a path to higher accuracy with manageable human effort.

The exploration of the Agentic RAG integration with advanced techniques is a frontier for agentic systems research. The combination of LightRAG architecture with agentic systems, as implemented in the PoC system described by Midokura (2025), could provide more efficient knowledge graph construction and update. The application of the Framework to additional domains and use cases will provide valuable insights into the generalizability of the approach.

6. Conclusion

This research has investigated the application of GraphRAG and knowledge grounding techniques to enhance the reliability and effectiveness of agentic systems in SAP environments. We have presented a comprehensive framework comprising a Knowledge Graph Construction Pipeline, a GraphRAG Retrieval Engine, a Multi-Agent Grounding Framework, and a Continuous Learning Module. The framework addresses the fundamental challenge of grounding LLM-based agents in the complex, relationship-rich semantic landscape of enterprise SAP systems.

The contribution of this research is threefold. First, we have demonstrated a practical approach to constructing enterprise knowledge graphs from SAP metadata and business process definitions, including ontology definition, triple extraction, and canonicalization. The knowledge graph provides a structured representation that captures the semantic relationships between tools, parameters, business objects, and organizational units. Second, we have developed and evaluated a GraphRAG retrieval engine that combines graph-based traversal with vector-based similarity search to provide comprehensive context for agent reasoning. The Ensemble of Ego Graphs approach improves tool retrieval for multi-step tasks by identifying tool chains that would be missed by vector-only retrieval. Third, we have designed and validated a Multi-Agent Grounding Framework that orchestrates specialized agents (Planner, Executor, Verifier, Learning) with a grounding protocol to ensure that all agent reasoning is anchored in verifiable facts.

The empirical evaluation of the framework has demonstrated significant improvements over baseline approaches. The GraphRAG-enhanced system achieved 94.8% accuracy compared to 65.2% for basic RAG and 78.4% for vector-only retrieval. The traceability and hallucination metrics showed particularly substantial improvements, with Grounding Completeness increasing from 62.1% to 89.7% and hallucination rates decreasing from 14.3% to 2.8%. The continuous learning module contributed a 5.5 percentage point improvement over 8 weeks of operation, demonstrating the value of feedback mechanisms for sustained system quality.

The implications of this research for enterprise AI practice are significant. Organizations deploying AI agents in SAP environments should prioritize the construction of enterprise knowledge graphs as foundational infrastructure, adopt a multi-agent architecture with specialized agent roles, implement continuous learning mechanisms, and establish grounding protocols that ensure all reasoning is anchored in verifiable facts. The investment in these capabilities will yield dividends in improved accuracy, reduced hallucination, enhanced traceability, and sustained performance improvement.

Future research should extend the framework to additional SAP modules and enterprise systems, develop automated ontology evolution mechanisms, improve LLM-based triple extraction accuracy, and explore integration with advanced techniques such as LightRAG. The continued evolution of GraphRAG and agentic systems will enable increasingly sophisticated and reliable AI automation in enterprise environments, transforming how organizations leverage their SAP investments.

References

1. Bansal, S., Sistla, S. S., Arikatala, A., & Schreiber, S. (2025). Planning agents on an ego-trip: Leveraging hybrid ego-graph ensembles for improved tool retrieval in enterprise task planning. *arXiv preprint arXiv:2508.05888*.

2. Berti, A., van der Aalst, W. M. P., & Adams, J. (2024). Enabling natural language analysis for object-centric event logs. *Springer*.

3. Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*. https://doi.org/10.48550/arXiv.2312.10997

4. Koroteev, M. (2021). BERT: A review of applications in natural language processing and understanding. *arXiv preprint arXiv:2103.11943*. https://doi.org/10.48550/arXiv.2103.11943

5. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural*

Information Processing Systems, *33*, 9459–9474. https://doi.org/10.48550/arXiv.2005.11401

6. Midokura. (2025). Grounding AI agents with domain knowledge: Reducing hallucination risk with knowledge graphs. *Midokura Technical Report*.

7. Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*. https://doi.org/10.48550/arXiv.1301.3781

8. MahendraVardhan, A., & Sridhar, S. (2022, December). Determining False Positive Analysis of Software Vulnerabilities with Predefined Scan Rules using Random Forest Classifier and Decision Tree Technique. In *2022 4th International Conference on Advances in Computing, Communication Control and Networking (ICAC3N)* (pp. 622-625). IEEE. https://doi.org/10.1109/ICAC3N56670.2022.10074458

9. Amilineni, M. V. (2025). Audit-Safe Agentic RAG Framework for Regulated Enterprise Systems: A Trustworthy AI Architecture for SAP, Finance, Healthcare, and Government Workflows. *International Journal of Advances in Signal and Image Sciences*, 34-46. https://doi.org/10.29284/7fjzk883

10. Ness, S. (2024). VAT/GST harmonisation challenges for digital assets such as bitcoin and NFTs in the EU following Case C-264/14 (Skatteverket v David Hedqist). *International Cybersecurity Law Review*, *5*(3), 459-490. https://doi.org/10.1365/s43439-024-00124-2

11. Ness, S. (2025). Securing Networks Against Adversarial Domain Name System Tunneling Attacks Using Hybrid Neural Networks. *IEEE Access*. https://doi.org/10.1109/ACCESS.2025.3550853

12. Neo4j. (2025). Power connected intelligence for AI and analytics with Neo4j and Databricks. *Neo4j Blog*.

13. SAP Learning. (2025). Exploring SAP knowledge graph. *SAP Learning Course*.

14. SAP RAG-Min. (2025). SAP RAG Minimal: A lightweight RAG tool for SAP documents. *GitHub Repository*.

15. Van der Aalst, W. M. P. (2022). Process mining: Data science in action (2nd ed.). *Springer*.

16. Van der Aalst, W. M. P. (2023). Object-centric process mining: An overview. *Springer*.

17. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30. https://doi.org/10.48550/ARXIV.1706.03762